

BANKING MANAGEMENT SYSTEM

Simple Python FastAPI Full-Stack Assignment

PROJECT SCENARIO

Develop a simple web application for a small bank. Bank staff should be able to create and manage customer accounts, perform deposits and withdrawals, and view transaction history.

1. Learning Objective

Use Python FastAPI to build REST APIs, SQLAlchemy with SQLite for persistence, Pydantic for validation, and plain HTML, Bootstrap and JavaScript for the frontend. The focus is correct business logic, not advanced banking security.

2. Required Modules

Account Management	Create, view, search, update and close a bank account.
Deposit	Add money to an active account and update its balance.
Withdrawal	Deduct money only when sufficient balance is available.
Transactions	Record every deposit and withdrawal with date, amount and type.
Dashboard	Display total accounts, active accounts and combined balance.

3. Suggested Account Information

account_number	Unique, automatically generated or entered by the student.
customer_name	Required; 3-100 characters.
email	Required and valid format.
phone	Required; 10 digits.
account_type	Savings or Current.
balance	Zero or positive amount.
status	Active or Closed.
created_date	Automatically stored.

IMPORTANT RULE

A closed account cannot accept deposits or withdrawals. Account number must not be changed after creation.

Functional and Submission Requirements

4. Business Rules

- Opening balance must be zero or greater.
- Deposit amount must be greater than zero.
- Withdrawal amount must be greater than zero.
- Withdrawal must fail when the balance is insufficient.
- The balance must never become negative.
- Every successful deposit or withdrawal must create a transaction record.
- A missing account must return a clear 404 error.
- Invalid request data must return validation details.

5. Expected REST APIs

POST	/api/accounts	Create account
GET	/api/accounts	View all accounts
GET	/api/accounts/{number}	View one account
PUT	/api/accounts/{number}	Update customer details
DELETE	/api/accounts/{number}	Close account
POST	/api/accounts/{number}/deposit	Deposit money
POST	/api/accounts/{number}/withdraw	Withdraw money
GET	/api/accounts/{number}/transactions	View history

6. Frontend Requirements

- Account registration form • Search box • Account table • Deposit and withdrawal forms • Transaction-history section • Success/error alerts • Confirmation before closing an account

7. Submission

Source files	Python files, static/index.html, requirements.txt, .gitignore and sample .env.
Evidence	Swagger screenshots and screenshots of the working frontend.
Testing	At least 8 test cases, including insufficient balance and invalid amount.
Explanation	A 2-3 minute demonstration of request flow and business rules.

8. Evaluation - 20 Marks

Account CRUD	5
Deposit/withdrawal logic	5
Validation and errors	4
Frontend integration	3
Transaction history	2
Code clarity	1

OPTIONAL BONUS

Add an AI banking help assistant that explains general application features. It must not give financial advice, approve loans or perform transactions.